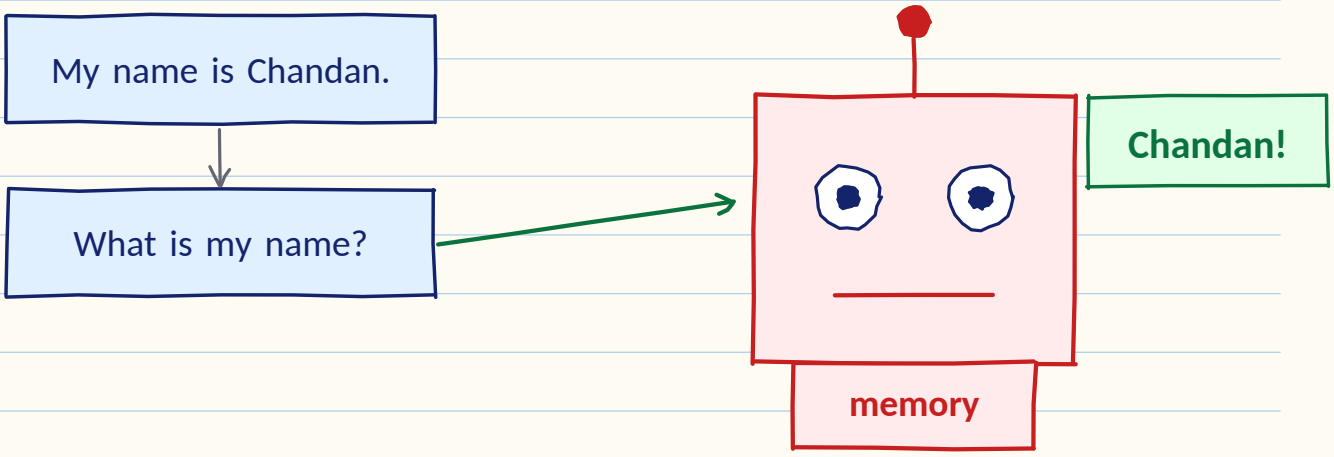
AI Agent Engineering

Day 02

Conversation Memory for an LLM Chat Assistant

LangChain + ChatOpenAI + Ollama + Qwen3:1.7B

From intuition and math to complete code logic



history = [Human, AI, Human, AI, ...] -> model -> answer

Handwritten Study Notes

Author: Chandan Chaudhari

GitHub: chandanc5525

github.com/chandanc5525/AI_Agent_Engineering_Sessions

Contents

1. Day 01 Recap and Day 02 Goal 3
2. Why LLMs Forget: The Stateless Function 4
3. Memory as Conditioning (Probability View) 5
4. How a Chat Becomes Tokens 6
5. Cost of Memory: Growth Math 8
6. Context Window Budget and Truncation 10
7. Attention Over Past Turns 11
8. Memory Strategies Compared 13
9. Message Types and Roles Revisited 15
10. Setup: .env and the OpenAI-Compatible Client 16
11. Code Walkthrough: Imports and Client 17
12. The Memory List and the Loop 19
13. Program Flow and State Trace 21
14. Execution Trace and Output 22
15. Review: Pitfalls in My Code 23
16. Improved Assistant: Window, System Prompt, Rollback 25
17. Persisting Memory to Disk 27
18. The LangChain Way: Templates and History 28
19. Streamlit Version with session_state 30
20. Testing Memory: Experiments 32
21. Troubleshooting Guide 33
22. Interview Q and A 35
23. Formula Sheet and Glossary 36
24. Exercises, Checklist, Day 03 Preview 38

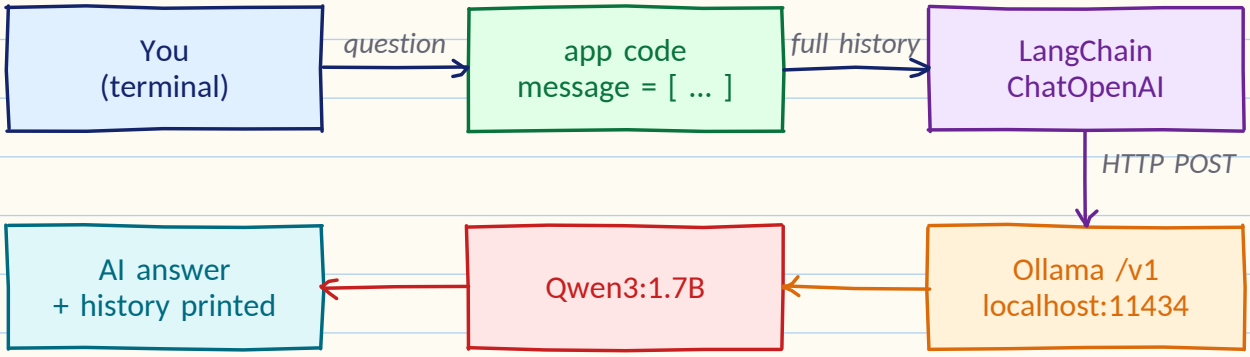
How to read these notes

Sections 1-9: intuition and math of memory (PyTorch and TensorFlow code). Sections 10-15: my code, line by line. Sections 16-19: improved versions. Sections 20-24: revision.

1. Day 01 Recap and Day 02 Goal

Day 01 built a one-shot assistant: question in, answer out. Each invoke was an independent request, so the model could not recall what I said one message earlier. Day 02 fixes exactly that single weakness: I keep a list of the conversation and send the whole list with every call.

	Day 01	Day 02
Goal	Call the LLM once per question	Hold a conversation
Payload sent	[HumanMessage]	[Human, AI, Human, AI, ..., Human]
Client	ChatOllama (native API)	ChatOpenAI (OpenAI-compatible /v1)
New classes	HumanMessage	AIMessage (plus the history list)
New concept	Stateless call	Memory is client-side state



After Day 02 I should be able to ...

- explain why an LLM forgets and why memory is an application feature, not a model feature
- write the history-list pattern with HumanMessage and AIMessage
- compute how history inflates token cost and fits (or overflows) the context window
- choose between buffer, window, summary and retrieval memory
- explain why ChatOpenAI needs the /v1 URL while ChatOllama must not have it

One-line summary

Memory = a Python list that I resend every time. The model itself is still a stateless function; I simulate a conversation by showing it the transcript.

2. Why LLMs Forget: The Stateless Function

Seen from outside, a chat model is a pure function. The same input always defines the same output distribution; nothing is stored inside between two HTTP calls.

reply = f(messages) (no hidden state between calls)

the model weights are frozen; only the input changes

Day 01: turn 2

payload sent to the model:

H: What is my name?

AI: I don't know.

Day 02: turn 2

payload sent to the model:

H: My name is Chandan.

A: Nice to meet you!

H: What is my name?

AI: Your name is Chandan.

So where does memory live?

- Not inside the model weights (training is finished).
- Not on the Ollama server between requests (it only caches computed key/value tensors for speed).
- In my program: the list message in RAM. If the program exits, the memory is gone.

The illusion of a conversation

When ChatGPT "remembers" the first message, it is simply receiving the first message again, silently, inside every new request. Every chat product is this trick plus clever trimming, summarising and storage.

Interview line

"LLMs are stateless. Conversation state is maintained by the client or orchestration layer, which resends the relevant history on each call."

3. Memory as Conditioning (Probability View)

Day 01 math: the model returns P(next token | context). On Day 02 the context grows to include every earlier turn, so the same formula now conditions on the whole transcript. Memory is just conditioning on more text.

$$P(a_t \mid u_1, a_1, \ldots, u_{t-1}, a_{t-1}, u_t)$$

u = user messages, a = assistant answers; Day 01 conditioned on u_t only

Why more context changes the answer

Question: "What is my name?". Without history the probability of the token "Chandan" is tiny (the model cannot know). With the history "My name is Chandan." the probability jumps. Illustrative numbers:

Context	P("Chandan")	Loss = -ln P
Day 01 (question only)	0.01	4.605 (very surprised)
Day 02 (with history)	0.782	0.246 (not surprised)

$$-\ln(0.01) = 4.605 \quad -\ln(0.782) = 0.246$$

cross-entropy: lower is better; memory lowers it for questions about earlier facts

PyTorch: loss with and without memory

```python
import torch

p_without, p_with = torch.tensor(0.01), torch.tensor(0.782)
print(-torch.log(p_without).item())   # 4.6052
print(-torch.log(p_with).item())      # 0.2459
```

TensorFlow: loss with and without memory

```python
import tensorflow as tf

p_without, p_with = tf.constant(0.01), tf.constant(0.782)
print(-tf.math.log(p_without).numpy())   # 4.6052
print(-tf.math.log(p_with).numpy())      # 0.2459
```

Intuition

The model is not "remembering". It is predicting the most likely continuation of a transcript that happens to contain the answer. Remove the transcript and the answer disappears.

4. How a Chat Becomes Tokens

The model reads one flat sequence of tokens, not a Python list. Before inference, the server applies the model's chat template, turning each role-tagged message into special marker tokens. Qwen models use a ChatML-style format (roughly as below; exact details depend on the model version).

<|im_start|>system You are a concise assistant.

<|im_start|>user My name is Chandan.

<|im_start|>assistant Nice to meet you, Chandan!

<|im_start|>user What is my name?

<|im_start|>assistant ... (model continues here)

flattened into one token sequence

- Each message is wrapped between role markers such as <|im_start|>user and <|im_end|>.
- The final marker for the assistant is left open: the model completes it token by token.
- Ollama applies this template for me. With /v1/chat/completions I send structured messages; I never build the text myself.
- Roles in LangChain map to roles in the template: SystemMessage = system, HumanMessage = user, AIMessage = assistant.

Consequences

1. The template markers also cost tokens, so a chat is slightly bigger than the plain text.
2. The model sees my past assistant messages as its own earlier words. If one contains a mistake, it may build on it.
3. Whatever I put into the list is what the model "remembers", including anything I accidentally add.

Check

An AIMessage in history should contain exactly what the user saw as the answer. If hidden reasoning text leaks into it (Qwen3 think blocks), the model will re-read it on every later turn.

5. Cost of Memory: Growth Math

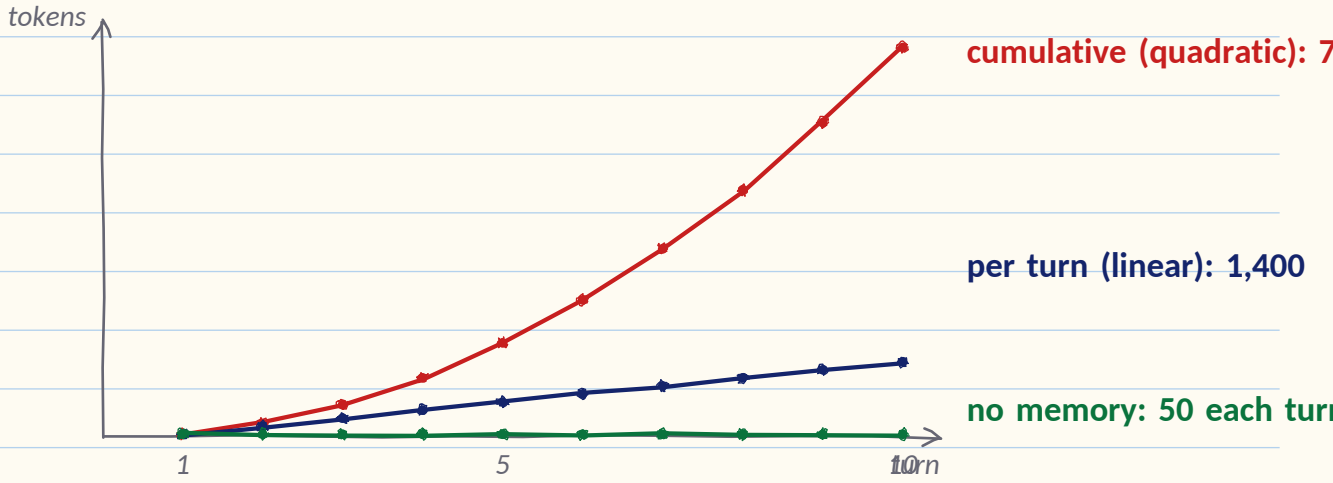
Resending the transcript is not free. Every turn pays for all previous turns again. Let one question cost q = 50 tokens, one answer a = 100 tokens, so one full exchange adds t = q + a = 150 tokens.

1. Turn 1 sends 50 tokens.
2. Turn 2 sends 150 (history) + 50 (new question) = 200 tokens.
3. Turn i sends (i - 1) × 150 + 50 tokens.

$$T(n) = n \cdot q + t \cdot n(n - 1) / 2$$

total input tokens processed over n turns (arithmetic series: growth is quadratic)

For n = 10: T = 10×50 + 150×45 = 7,250 tokens, compared with only 500 without memory (14.5× more). For n = 20 it is 29,500; for n = 50 it is 186,250.



PyTorch: input tokens per turn and total

```
import torch

q, a, n = 50, 100, 10
t = q + a
turns = torch.arange(1, n + 1)
prompt = (turns - 1) * t + q          # tokens sent at each turn
print(prompt.tolist())                # [50, 200, ..., 1400]
print(prompt.sum().item())            # 7250
print(n * q)                          # 500 without memory
```

TensorFlow: input tokens per turn and total

```
import tensorflow as tf

q, a, n = 50, 100, 10
t = q + a
turns = tf.range(1, n + 1)
prompt = (turns - 1) * t + q
print(prompt.numpy())                    # [50 200 ... 1400]
print(tf.reduce_sum(prompt).numpy())     # 7250
```

What grows besides tokens?

- Latency: time to first token is prompt processing (prefill). Illustration: 1,500 prompt tokens at 300 tokens/s prefill = 5 s; then 150 answer tokens at 20 tokens/s = 7.5 s.
- Memory: the KV cache grows with history. With the illustrative 112 KB per token from Day 01, 1,500 tokens is about 172 MB.
- Attention work: each prefill compares every token with every earlier one, so it scales roughly with the square of the length.

latency ≈ prompt_tokens / prefill_speed + output_tokens / decode_speed

both terms grow as the conversation gets longer

6. Context Window Budget and Truncation

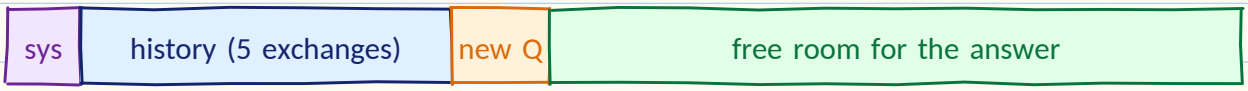
The context window C is the maximum number of tokens the model can read at once (prompt plus the answer it writes). The history must fit inside it, with room left for the reply.

$$k_{max} = \lfloor (C - S - R) / t \rfloor$$

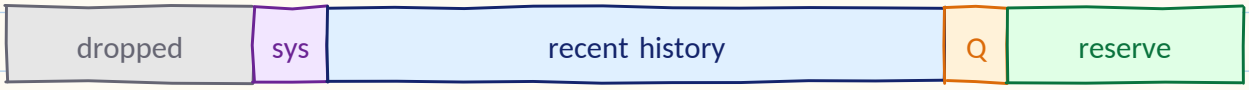
C = window, S = system prompt tokens, R = reserved for the answer, t = tokens per exchange

Example: C = 4096, S = 50, R = 512, t = 150. Then (4096 - 50 - 512) / 150 = 23.56, so about 23 exchanges fit. The 24th pushes something out.

Turn 5: everything fits



Turn 30: oldest turns must go



What happens when it overflows?

- Depending on the server and settings, the oldest tokens are dropped (the model silently forgets the beginning) or the request is truncated or fails.
- The model never tells me it forgot. It just answers as if the early facts never existed.
- Ollama's default context is small (a few thousand tokens, version dependent). As far as I know, the OpenAI-compatible /v1 endpoint cannot set `num_ctx` per request; set it in a Modelfile or the server configuration instead.

Estimating tokens before sending

- Rule of thumb: 1 token is about 4 English characters (0.75 words).
- Better: read usage_metadata from the previous response (input_tokens and output_tokens) and add them up.
- Cheap guard: stop or trim when estimated tokens exceed about 75% of the window.

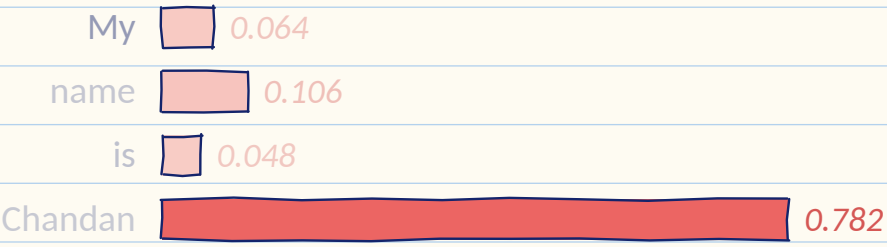
Design rule

Always budget: window = system + history + new question + reserved answer. Trim history, never the reserve.

7. Attention Over Past Turns

Why can the model answer "What is my name?" from a message sent three turns earlier? Because of causal self-attention (Day 01, Section 7). Each new token forms a query and compares it with the keys of all earlier tokens, including those from old turns.

Query token: "name?" (end of the new question)



(illustrative attention weights, one head)

Suppose the raw similarity scores between the query and the history keys My, name, is, Chandan are [0.5, 1.0, 0.2, 3.0]. Softmax turns them into weights:

softmax([0.5, 1.0, 0.2, 3.0]) = [0.064, 0.106, 0.048, 0.782]

about 78% of the attention goes to "Chandan", so its value vector flows into the answer

PyTorch: attention weights over history

```
import torch

scores = torch.tensor([0.5, 1.0, 0.2, 3.0])
w = torch.softmax(scores, dim=0)
for tok, p in zip(["My", "name", "is", "Chandan"], w):
    print(tok, round(p.item(), 3))
```

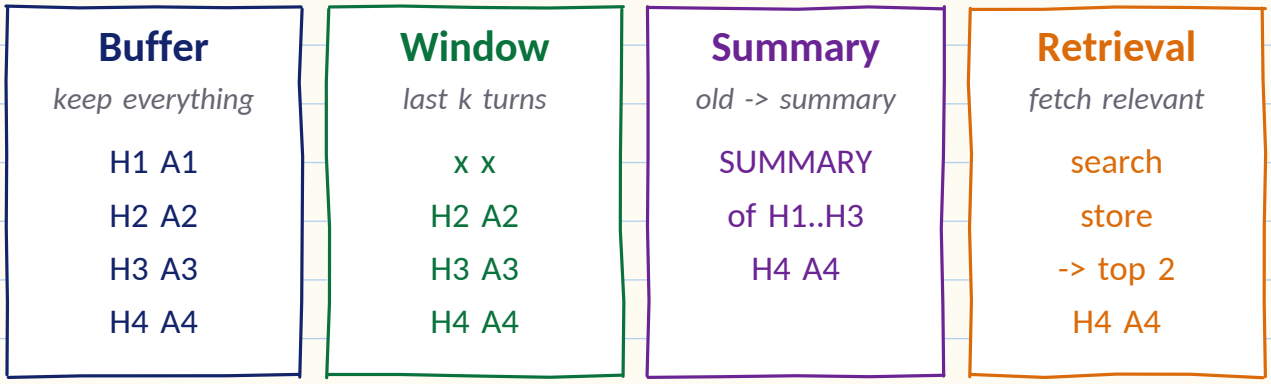
TensorFlow: attention weights over history

```
import tensorflow as tf

scores = tf.constant([0.5, 1.0, 0.2, 3.0])
w = tf.nn.softmax(scores)
for tok, p in zip(["My", "name", "is", "Chandan"], w.numpy()):
    print(tok, round(float(p), 3))
```

- Long histories dilute attention: softmax weights must sum to 1, so with thousands of tokens each one gets a smaller share. Small models like 1.7B can miss facts buried deep in a long chat.
- KV cache: keys and values of old turns are computed once and reused. Many servers reuse the cached prefix when the next request starts with the same history, which saves prefill time.
- Order matters: put stable instructions (system prompt) first so the shared prefix stays identical.

8. Memory Strategies Compared



Strategy	How it works	Pros	Cons
Buffer (full)	Send the whole list (my Day 02 code)	Perfect recall, simplest	Tokens grow quadratically; hits the window
Window (last k)	Keep the most recent k exchanges	Bounded cost	Forgets early facts such as the name
Token-trimmed	Drop oldest until under a token budget	Fits the window exactly	Needs a token counter
Summary	LLM compresses old turns into notes	Long memory, small size	Extra LLM call; can lose detail
Retrieval	Embed turns, fetch relevant ones	Scales to huge history	More infrastructure (vector DB)

compression ratio = original_tokens / summary_tokens

example: 2,000 tokens of old chat -> 200-token summary = 10×

Window memory with trim_messages

Window by message count

```
from langchain_core.messages import trim_messages

trimmed = trim_messages(
    history,
    strategy="last",        # keep the most recent messages
    token_counter=len,      # simple: count messages, not tokens
    max_tokens=16,          # last 16 messages = 8 exchanges
    start_on="human",       # never start with an orphan AI reply
    include_system=True,    # keep the SystemMessage
)
```

Summary memory (sketch)

Summarise old turns

```
def summarise(old_messages, llm):
    text = "\n".join(f"{m.type}: {m.content}" for m in old_messages)
    ask = "Summarise this chat in 5 short bullets:\n" + text
    return llm.invoke(ask).content

# history = [SystemMessage("Summary: " + summary)] + recent_messages
```

Which one do I pick?

Start with a window of the last 6-10 exchanges. Add a summary if early facts matter. Use retrieval only when the history is far bigger than any window.

9. Message Types and Roles Revisited

Class	Role	Who writes it	In my Day 02 code
SystemMessage	system	Developer	Not used yet (added in Section 16)
HumanMessage	user	Person typing	Appended after every input()
AIMessage	assistant	Model	Appended after every reply
ToolMessage	tool	Tool result	Later days (agents)

Rules of a healthy history

1. Optional SystemMessage first. It defines behaviour and does not change during the chat.
2. Then strictly alternate Human, AI, Human, AI ... (some providers reject two Humans in a row).
3. The list must end with a HumanMessage when I call the model. The model writes the next AI turn.
4. Never append an AIMessage unless it was really produced (or you intentionally pre-fill an example).

Useful AIMessage facts

- response returned by invoke is already an AIMessage.
- My code rebuilds it with AIMessage(content=response.content). That keeps only the text. The alternative message.append(response) also keeps usage_metadata and response_metadata (token counts, finish reason).
- msg.type returns "human", "ai" or "system"; isinstance(msg, HumanMessage) does the same check in my display loop.

A hand-written history (few-shot style)

```
from langchain_core.messages import (
    SystemMessage, HumanMessage, AIMessage)

history = [
    SystemMessage(content="You are a concise assistant."),
    HumanMessage(content="My name is Chandan."),
    AIMessage(content="Nice to meet you, Chandan!"),
    HumanMessage(content="What is my name?"),
]
reply = llm.invoke(history)   # AIMessage: Your name is Chandan.
```

Pre-filled history

Because the model treats the list as its own past, I can pre-load fake AIMessages as examples to steer style (few-shot prompting). That is the same mechanism as memory.

10. Setup: .env and the OpenAI-Compatible Client

Day 02 uses the OpenAI-compatible client, so the .env from the README works unchanged, including the /v1 suffix that broke ChatOllama on Day 01.

.env

```
BASE_URL=http://localhost:11434/v1
API_KEY=ollama
MODEL=qwen3:1.7b
```

Client	Package	base_url	Why
ChatOllama (Day 01)	langchain-ollama	http://localhost:11434	Native API, path /api/chat
ChatOpenAI (Day 02)	langchain-openai	http://localhost:11434/v1	OpenAI-style path /v1/chat/completions

Terminal: setup and run

```
uv add langchain-openai python-dotenv   # langchain-core comes along
ollama pull qwen3:1.7b                  # if not downloaded yet
uv run python main.py                   # run the terminal assistant
```

- API_KEY=ollama is a dummy. Ollama checks nothing, but the OpenAI client library refuses an empty key.
- Pointing ChatOpenAI at Ollama means the same code can later target real OpenAI by changing three variables only.
- .env stays out of Git (.gitignore). Only real secrets matter, but the habit is what counts.
- If I see a 404 now, check that /v1 is present; on Day 01 the mistake was the opposite.

Memory aid

ChatOllama = host only. ChatOpenAI = host + /v1. Wrong door, 404.

11. Code Walkthrough: Imports and Client

My complete Day 02 program (comments shortened). The following sections explain it part by part.

main.py (my Day 02 code)

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage

load_dotenv()

model_name = os.getenv("MODEL", "qwen3:1.7b")
base_url = os.getenv("BASE_URL", "http://localhost:11434/v1")
api_key = os.getenv("API_KEY", "ollama")

llm = ChatOpenAI(
```

```
    model=model_name,
    base_url=base_url,
    api_key=api_key
)

print("*" * 40)
print("My LangChain Ollama Assistant")
print("*" * 40)

message = []                          # conversation memory

while True:

    user_input = input("\nYou: ")

    if user_input.lower() == "exit":
        print("Exiting the assistant. Goodbye!")
        break

    message.append(HumanMessage(content=user_input))
    response = llm.invoke(message)
    message.append(AIMessage(content=response.content))

    print("\nAI:", response.content)

    print("\n" + "=" * 40)
    print("CONVERSATION HISTORY")
    print("=" * 40)

    for msg in message:
        if isinstance(msg, HumanMessage):
            print("\nYou:", msg.content)
        elif isinstance(msg, AIMessage):
            print("\nAI:", msg.content)

    print("=" * 40)
```

Imports: what changed from Day 01

- `ChatOpenAI` from `langchain_openai` replaces `ChatOllama`: a general client for any OpenAI-style server.
- `AIMessage` is new. I need it to store the model's replies in the history.
- `HumanMessage`, `os` and `load_dotenv` are the same as before.

Configuration and client

- `load_dotenv()` first, then `os.getenv(name, default)` for MODEL, BASE_URL and API_KEY.
- Defaults make the script runnable even without a `.env`.
- `ChatOpenAI(model=..., base_url=..., api_key=...)` creates the client object. No network call yet; the first request happens at `invoke`.
- Note: ChatOpenAI reads `base_url` and `api_key` explicitly here. It can also read OPENAI_API_KEY from the environment, but passing values keeps the code transparent.

12. The Memory List and the Loop

The memory list

Step 1

```
message = []                                # empty at the start
```

One plain Python list, living in RAM, outside the model. It will hold alternating `HumanMessage` and `AIMessage` objects. (I named it `message`; the plural `messages` or `history` reads better.)

One loop iteration, step by step

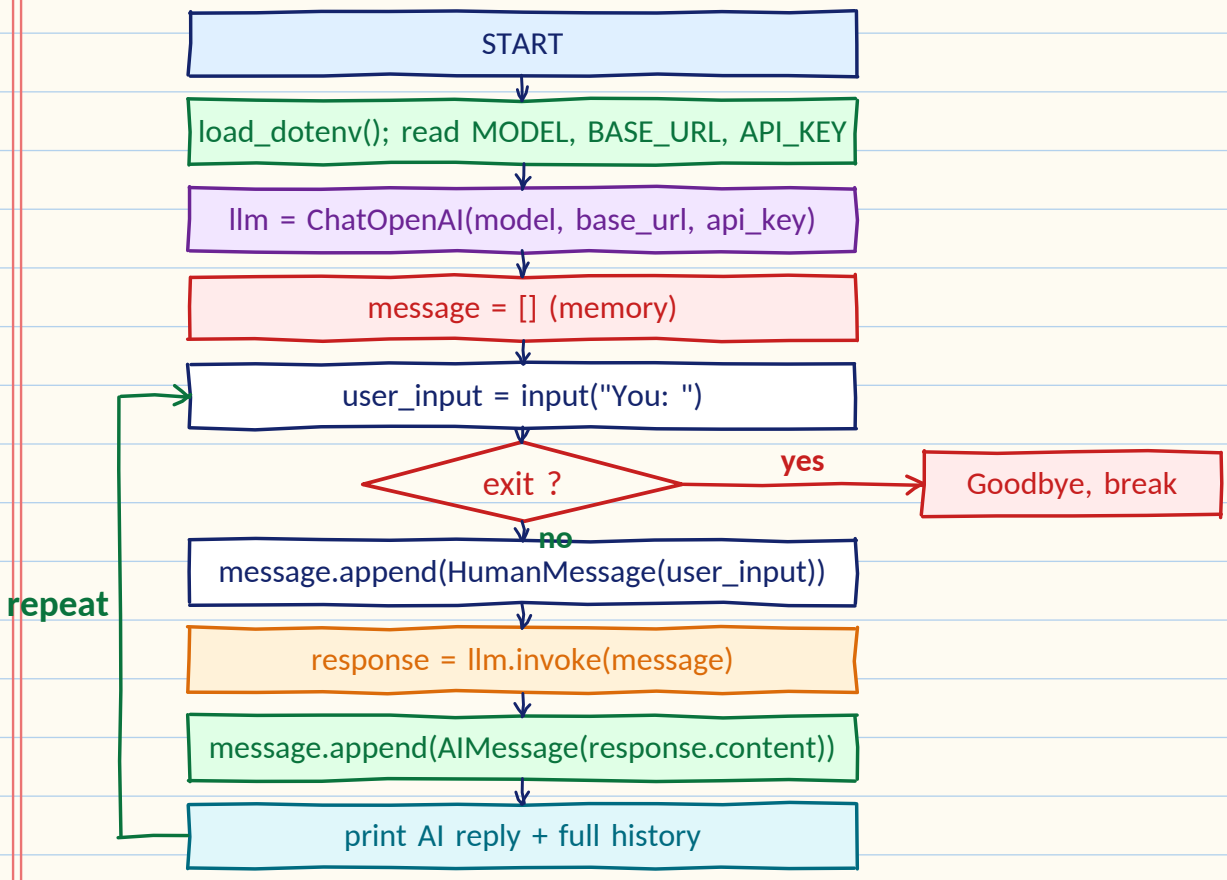
1. `input("\nYou: ")` waits for a line of text. `\n` only adds a blank line.
2. `.lower()` `==` `"exit"` makes the exit check case-insensitive, then `break` leaves the loop.
3. `message.append(HumanMessage(content=user_input))` adds the new question before the call. The list now ends with a Human message.
4. `llm.invoke(message)` sends the entire list. LangChain converts each object to {"role", "content"} JSON for /v1/chat/completions.
5. `message.append(AIMessage(content=response.content))` stores the answer after the call. Now the list ends with an AI message again.
6. `print("\nAI:", response.content)` shows the reply to the user.
7. The `for` `msg` `in` `message` loop prints the whole transcript, using `isinstance` to label each entry You or AI.

Invariants I rely on

- Before `invoke`: list length is odd (ends with Human).
- After the append of the reply: length is even (complete exchanges).
- Length after n turns = 2n messages.

> **Order matters**
> Append the question first, call the model second, append the answer third. If I appended the question after the call, the model would never see it.

13. Program Flow and State Trace



State trace of the list

H = HumanMessage, A = AIMessage. The list grows by two entries per turn, and the model always receives everything.

start	[ ] (empty list)	*len = 0*
after turn 1	H1 A1	*len = 2*
after turn 2	H1 A1 H2 A2	*len = 4*
after turn 3	H1 A1 H2 A2 H3 A3	*len = 6*

Reading the picture

Turn 3 sends H1 A1 H2 A2 H3 (5 messages), the model replies A3, and I store it. So at call time the list length is 2n - 1.

14. Execution Trace and Output

Sample session (wording will differ)

```
$ uv run python main.py
****************************************
My LangChain Ollama Assistant
****************************************

You: My name is Chandan.

AI: Nice to meet you, Chandan! How can I help?

========================================
CONVERSATION HISTORY
========================================

You: My name is Chandan.

AI: Nice to meet you, Chandan! How can I help?
========================================

You: What is my name?

AI: Your name is Chandan.

========================================
CONVERSATION HISTORY
========================================

You: My name is Chandan.
AI: Nice to meet you, Chandan! How can I help?
You: What is my name?
AI: Your name is Chandan.
========================================

You: exit
Exiting the assistant. Goodbye!
```

- Turn 2 proves memory works: the model answers a question whose answer exists only in the first message.

- The history block is printed after every turn, so the output grows quadratically too (a debugging aid, noisy for real users).
- Latency also grows: later turns are slower because the prompt is longer (Section 5).
- Qwen3 may print extra reasoning text depending on the server version. Section 15 covers it.

Experiment to run

1. Tell the model three facts, then ask about the first one. Does it recall it?
2. Restart the program and ask again. The memory is gone (it was only in RAM).
3. Time turn 1 and turn 10 with the same short question. Compare.

15. Review: Pitfalls in My Code

The code works and teaches the core idea well. These are the weak spots I should know, with fixes that appear in the improved version (Section 16).

#	Issue	Why it matters	Fix
1	History is unbounded	Tokens grow quadratically; eventually exceeds the window and early turns are lost	Window or trim
2	No SystemMessage	No stable instructions or persona	Add SystemMessage first
3	Failure leaves a dangling Human message	If invoke raises, the list ends with Human; the next call has two Humans	try/except and pop()
4	Empty input is appended	Wastes tokens, confuses the model	Strip and skip empty
5	Qwen3 think text may be stored	Reasoning is re-sent every turn, bloating context	strip_think() before append
6	Memory lost on exit	RAM only	Save to JSON (Section 17)

7	"exit" compared without strip()	" exit" with a space is treated as a question	input().strip()
8	History printed every turn	Noisy, quadratic output	Print only on a command
9	Rebuilds AIMessage from content	Drops usage metadata	append(response) if needed
10	Variable named message	Reads like a single message	history or messages

Think blocks and memory

Qwen3 can emit a reasoning block before its answer. If that text ends up inside response.content, my code stores it and resends it on every later turn. Depending on the server version, reasoning may arrive in a separate field instead, so check once with a print before deciding. A defensive cleaner:

Remove <think> blocks

```
import re

def strip_think(text: str) -> str:
    pat = r"<think>.*?</think>"
    return re.sub(pat, "", text, flags=re.DOTALL).strip()
```

Rollback pattern

Add the Human message, try the call, and on any exception pop() the Human message so the list stays consistent (alternating, ending with an AI reply).

16. Improved Assistant: Window, System Prompt, Rollback

Same idea, made safer: system prompt, windowed memory, empty-input guard, rollback on error, think-block cleaning, a clear command and a clean main(). (Disk persistence is added in Section 17.)

main.py (improved memory assistant)

```
import os
import re
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import (
    AIMessage, HumanMessage, SystemMessage)

load_dotenv()

llm = ChatOpenAI(
    model=os.getenv("MODEL", "qwen3:1.7b"),
    base_url=os.getenv("BASE_URL", "http://localhost:11434/v1"),
    api_key=os.getenv("API_KEY", "ollama"),
    temperature=0.3,
)

SYSTEM = SystemMessage(content="You are a concise assistant.")
MAX_TURNS = 8                        # remember the last 8 exchanges


def strip_think(text: str) -> str:
    pat = r"<think>.*?</think>"
    return re.sub(pat, "", text, flags=re.DOTALL).strip()


def window(history):
    return history[-2 * MAX_TURNS:]    # last N Human+AI pairs


def main() -> None:
    history = []
    print("Type 'exit' to quit, 'clear' to forget.")
    while True:
```

```
        q = input("\nYou: ").strip()
        if q.lower() == "exit":
            print("Goodbye!")
            break
        if q.lower() == "clear":
            history = []
            print("Memory cleared.")
            continue
        if not q:
            continue
        history.append(HumanMessage(content=q))
        try:
            reply = llm.invoke([SYSTEM] + window(history))
        except Exception as err:
            history.pop()               # roll back the question
            print(f"[error] {err}")
            continue
        answer = strip_think(reply.content)
        history.append(AIMessage(content=answer))
        print("\nAI:", answer)


if __name__ == "__main__":
    main()
```

What changed and why

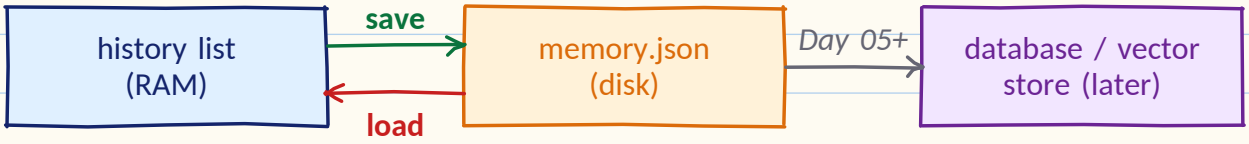
Change	Reason
SYSTEM + [SYSTEM] + window(...)	Instructions are always first and never trimmed away
history[-2 * MAX_TURNS:]	Bounded token cost; slides forward as the chat grows
try / except + pop()	A failed call leaves the history valid
strip_think()	Only the visible answer enters memory
"clear" command	Lets me reset context between topics
The full history is stored, the window is sent	I can still save or inspect everything

Trade-off

With MAX_TURNS = 8 the assistant forgets my name after 8 more exchanges. Real systems pin key facts into the system prompt or a summary. That is the next improvement.

17. Persisting Memory to Disk

A list in RAM disappears when the program ends. To remember across runs, I serialise the messages to a file (later: a database). LangChain provides helpers that turn message objects into plain dictionaries and back.



Save and load the message list

```
import json
import os
from langchain_core.messages import (
    messages_from_dict, messages_to_dict)

FILE = "memory.json"


def load_history():
    if os.path.exists(FILE):
        with open(FILE, encoding="utf-8") as f:
            return messages_from_dict(json.load(f))
    return []


def save_history(history):
    with open(FILE, "w", encoding="utf-8") as f:
        json.dump(messages_to_dict(history), f, indent=2)
```

Usage in main(): start with history = load_history(), and call

save_history(history) after each appended reply (and after clear, so the file is emptied too).

- messages_to_dict returns a list of {"type": ..., "data": {...}} dictionaries: plain JSON-friendly data.
- messages_from_dict rebuilds the right classes (HumanMessage, AIMessage, ...).
- Add memory.json to .gitignore. Conversations may contain personal information.
- Several users or sessions: use one file (or key) per session id, for example memory_<session>.json.
- Persisted memory still obeys the context window: load everything, but send only the windowed part.

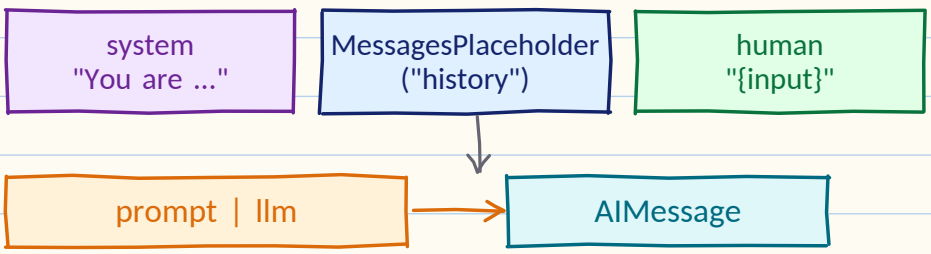
Privacy note

Stored chats can contain names, secrets and company data. Keep the file local, give users a way to clear it, and never commit it.

18. The LangChain Way: Templates and History

LangChain wraps the same pattern in reusable parts. Knowing the manual list first makes these easy to understand.

Prompt template with a history placeholder



Template + manual history

```
from langchain_core.prompts import (
    ChatPromptTemplate, MessagesPlaceholder)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise assistant."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
])
chain = prompt | llm            # LCEL: prompt feeds the model

q = "What is my name?"
reply = chain.invoke({"history": history, "input": q})
history += [HumanMessage(content=q), reply]
```

Automatic history per session

RunnableWithMessageHistory

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import (
    RunnableWithMessageHistory)

store = {}


def get_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]


chat = RunnableWithMessageHistory(
    chain, get_history,
    input_messages_key="input",
    history_messages_key="history",
)
cfg = {"configurable": {"session_id": "chandan"}}
print(chat.invoke({"input": "My name is Chandan."}, config=cfg))
print(chat.invoke({"input": "What is my name?"}, config=cfg))
```

- The wrapper loads the session's messages, injects them at the placeholder, calls the chain and stores both new messages.
- session_id separates users, which is exactly what a web backend needs.
- LangChain is moving long-term and agent state to LangGraph (checkpointers). The concept is the same: state + resend. Check the current docs before building on a specific class.

Why learn the manual version first

Frameworks change names often. The invariant does not: history in, model call, reply appended.

19. Streamlit Version with session_state

The README runs each day with uv run streamlit run app.py. Streamlit re-executes the whole script on every interaction, so a plain Python list would be reset each time. The fix is st.session_state, a dictionary that survives reruns. (Reference implementation; it is not a copy of the repo file.)

Day02/app.py (reference version)

```
import os
import streamlit as st
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage

load_dotenv()


@st.cache_resource
def get_llm():
    return ChatOpenAI(
        model=os.getenv("MODEL", "qwen3:1.7b"),
        base_url=os.getenv("BASE_URL", "http://localhost:11434/v1"),
        api_key=os.getenv("API_KEY", "ollama"),
    )
```

```
st.set_page_config(page_title="Day 02 - Memory")
st.title("Day 02: AI Agent with Memory")

if "history" not in st.session_state:
    st.session_state.history = []
history = st.session_state.history

if st.sidebar.button("Clear memory"):
    history.clear()

for m in history:
    role = "user" if isinstance(m, HumanMessage) else "assistant"
    with st.chat_message(role):
        st.write(m.content)

prompt = st.chat_input("Ask me anything...")
if prompt:
    with st.chat_message("user"):
        st.write(prompt)
    history.append(HumanMessage(content=prompt))
    with st.chat_message("assistant"):
        reply = get_llm().invoke(history)
        st.write(reply.content)
    history.append(AIMessage(content=reply.content))
```

How it works

- st.session_state.history is the Streamlit version of my message list: per browser session, survives reruns.
- Every rerun first redraws the old messages from the list, then handles the new input.
- st.cache_resource builds the client once instead of on every rerun.
- st.sidebar.button gives a Clear memory control (a rerun happens, and the list is emptied).
- Closing the browser tab or restarting the server clears memory; combine with Section 17 to persist.

Terminal vs web

Same logic, different storage: the terminal keeps the list in a variable, Streamlit keeps it in session_state. Keep model code separate from UI code.

20. Testing Memory: Experiments

Treat memory like a feature to test, not an assumption. Run these experiments and write down what happens.

Test	What to do	Expected result
Recall	Say your name, ask something else, then "What is my name?"	Correct name
Pronoun resolution	"Who wrote Hamlet?" then "When was he born?"	Understands "he"
Update	Say "My city is Mumbai", then "Actually it is Pune". Ask the city.	Pune (latest fact wins, usually)
Contradiction	Give two conflicting facts and ask	May hesitate or pick one; note it
Long-range	State a fact, chat 15 turns, ask for it	May be forgotten or blurred (small model)
Window cut	Set MAX_TURNS = 2 and repeat Recall	Name forgotten after 2 more turns
Clear	Type clear, then ask the name	Does not know
Persistence	Quit and restart with memory.json	Name still known

Measuring the cost of memory

Log token usage per turn

```
total_in = 0
for _ in range(10):
    reply = llm.invoke(history)
    meta = reply.usage_metadata or {}
    total_in += meta.get("input_tokens", 0)
    print(meta.get("input_tokens"), meta.get("output_tokens"))
print("input tokens so far:", total_in)
```

- Plot input_tokens per turn: it should rise roughly linearly (Section 5).
- If the numbers stop rising, the server is truncating the prompt (context overflow).
- Whether usage_metadata is filled depends on the client and server version; fall back to an estimate if it is empty.

Scientist mindset

A model that recalls the fact is not proof of understanding; a model that forgets is not proof of a bug. Check what was actually sent, then judge.

21. Troubleshooting Guide

Symptom	Likely cause	Fix
404 or "not found" from ChatOpenAI	base_url missing /v1	BASE_URL=http://localhost:11434/v1
Connection refused	Ollama not running	Start Ollama or run: ollama serve
model not found	Weights not pulled	ollama pull qwen3:1.7b
Missing credentials / api_key error	Empty API_KEY	Use any non-empty value, e.g. ollama
ModuleNotFoundError: langchain_openai	Package missing	uv add langchain-openai; run with uv run

Model forgets early facts	Window trimmed or context overflow	Raise the window, summarise, pin facts in system prompt
Replies get slower every turn	Prompt grows (prefill cost)	Window or summary; keep answers short
Strange reasoning text in answers	Qwen3 think output in content	strip_think() or /no_think in the prompt
Memory resets in Streamlit	Using a plain list	Use st.session_state
Two Human messages in a row	Failed call not rolled back	try/except with history.pop()
Answer ignores the history	Passed only the new message	Print len(history) before invoke

Debug checklist

1. Print `[m.type for m in history]`. Does it alternate and end with "human" before invoke?
2. Print the estimated or reported input tokens. Is it growing as expected?
3. Test the client in a tiny script with a hand-built history (Section 9).
4. Check that `ollama run qwen3:1.7b` works in the terminal.
5. Only then suspect the model quality.

22. Interview Q and A

Q1. How does a chatbot remember earlier messages?

It does not store them in the model. The application keeps the transcript and resends it (or a trimmed or summarised version) with every request.

Q2. Why are LLM APIs stateless?

Stateless servers scale easily and are simple: each request is independent and any replica can serve it. State is the client's responsibility.

Q3. What is the cost of conversation memory?

Tokens per request grow with history, so total tokens over n turns grow roughly quadratically, increasing cost, latency and KV-cache memory.

Q4. What is the context window and what happens when exceeded?

The maximum tokens (prompt plus answer) the model can process. Beyond it, the oldest content may be dropped, truncated or rejected, so the model silently loses early information.

Q5. Compare buffer, window and summary memory.

Buffer keeps everything (perfect recall, unbounded cost). Window keeps the last k turns (bounded, forgets early facts). Summary compresses old turns with the LLM (long memory, extra call, possible detail loss).

Q6. Where should a system prompt sit in the history?

First, and always preserved when trimming. It defines behaviour and keeps the cached prefix stable.

Q7. HumanMessage vs AIMessage vs SystemMessage?

User input, model output, and developer instructions. They map to the chat roles user, assistant and system.

Q8. How would you support many users?

Key the history by session id, store it in a database or cache (Redis), load and trim per request, and apply retention and privacy rules.

Q9. How do you persist memory?

Serialise messages (for example with messages_to_dict to JSON), store them per session, and reload at start. Long-term memory can use embeddings in a vector store.

Q10. Why does ChatOpenAI work with Ollama?

Ollama exposes an OpenAI-compatible /v1 API, so any OpenAI client can talk to it by changing base_url and using a dummy key.

Q11. Why do responses slow down in long chats?

The model must process the entire history again (prefill) before generating, and attention cost grows with length. Prefix caching helps but does not remove the growth.

Q12. How do you stop hidden reasoning from polluting memory?

Store only the final visible answer: strip think blocks, or use the model's non-thinking mode, before appending the AIMessage.

Interview tip

Connect the answer to numbers: "My 10-turn test sent 7,250 input tokens versus 500 without memory, so I added a window."

23. Formula Sheet and Glossary

Formula sheet

reply = f(messages)

stateless chat function: memory = the list I pass in

$$P(a_t \mid u_1, a_1, \ldots, u_t)$$

answer conditioned on the whole transcript

tokens at turn i = (i - 1)·t + q t = q + a

q = question tokens, a = answer tokens

$$T(n) = n \cdot q + t \cdot n(n - 1) / 2$$

total input tokens over n turns (quadratic); n = 10 gives 7,250

$$k_{max} = \lfloor (C - S - R) / t \rfloor$$

exchanges that fit: window C, system S, reserve R

softmax(scores) -> weights over history tokens

[0.5, 1.0, 0.2, 3.0] -> [0.064, 0.106, 0.048, 0.782]

loss = -ln P(correct token)

-ln 0.782 = 0.246 with memory; -ln 0.01 = 4.605 without

latency ≈ prompt/prefill_speed + output/decode_speed

5 s + 7.5 s in the 1,500-token example

Glossary

Term	Meaning
Stateless	No information is kept between requests
History / transcript	List of messages sent with each request
HumanMessage	User turn (role: user)
AIMessage	Model turn (role: assistant)
SystemMessage	Developer instructions (role: system)
Context window	Maximum tokens the model can read and write at once
Prefill	Processing the prompt before generating the first token
KV cache	Stored keys and values of earlier tokens
Window memory	Keep only the last k turns
Summary memory	Replace old turns by an LLM-written summary
Rollback	Undo the appended question when the call fails
session_state	Streamlit dictionary that survives reruns

OpenAI-compatible API Server that mimics /v1/chat/completions

24. Exercises, Checklist, Day 03 Preview

Practice exercises

1. Rename `message` to `history` in my code and re-run to confirm nothing breaks.
2. Append `response` instead of `AIMessage(content=response.content)` and print `msg.usage_metadata` for AI messages.
3. Add the rollback pattern (try/except with pop) and test it by stopping Ollama mid-chat.
4. Add a SystemMessage that makes the assistant answer like a Capgemini solution architect.
5. Set MAX_TURNS = 2 and design a test that proves the window forgets the name.
6. Log input tokens per turn for 15 turns and plot them with chart code in PyTorch or TensorFlow tensors.
7. Implement `summarise()` and replace turns older than 6 exchanges with one SystemMessage summary.
8. Persist to memory.json and verify that the name survives a restart.
9. Build the Streamlit version and add a token counter to the sidebar.

Day 02 completion checklist

[ ] langchain-openai installed; .env has the /v1 URL
[ ] Terminal assistant remembers earlier turns
[ ] I can draw the list state after turns 1, 2 and 3
[ ] I can compute T(n) and k_max for given numbers
[ ] Windowed and system-prompt version working
[ ] Memory saved to and loaded from JSON
[ ] Streamlit version with session_state running
[ ] I can explain why memory is client-side and why cost is quadratic

Day 03 preview: context-aware expert agent

From the README: Day 03 builds an agent that answers expert-level questions from one defined knowledge context. The idea builds directly on today: put the knowledge into the system message (or the prompt) and instruct the model to answer only from it. History tells the model what was said; the context tells it what is true.

Day 03 sketch: knowledge context in the system message

```
CONTEXT = "...policy, product notes or study material..."

SYSTEM = SystemMessage(content=(
    "You are a domain expert. Answer ONLY from the context below. "
    "If it is not there, say you do not know.\n\n" + CONTEXT))

reply = llm.invoke([SYSTEM] + window(history))
```

Final takeaway

The model is a stateless next-token function. Day 01 called it once; Day 02 wraps it in a list that remembers. Everything fancier (summaries, vector stores, agents) is a smarter way to decide what goes into that list.

- Chandan Chaudhari | github.com/chandanc5525 -